

運用 AlloyDB 和無伺服器執行階段，打造 AI 穿搭應用程式

來源：<https://codelabs.developers.google.com/smart-stylist-app?hl=zh-tw>

步驟數：16

在本程式碼研究室中，您將建構服裝推薦應用程式，提供配對服裝的智慧推薦內容。

目錄

1. 總覽
2. 架構
3. 事前準備
4. 資料庫設定
5. 資料擷取
6. 版本結構定義
7. 為情境建立嵌入
8. 執行向量搜尋
9. 使用 LLM 驗證比對結果
10. 將應用程式帶到網頁
11. 測試 retail-engine 應用程式
12. 瞭解服裝建議流程
13. 將應用程式帶到網路上
14. 測試服裝推薦應用程式
15. 清除所用資源
16. 恭喜

1. 總覽

想像一下，有了最先進的生成式 AI 整合技術，時尚應用程式不僅能協助你找到完美穿搭，還能提供即時造型建議！本次演講將探討如何運用 AlloyDB 的向量搜尋功能，搭配 Google 的 ScaNN 索引，建構出這類應用程式，以極快的速度搜尋相符的服裝，並即時提供時尚建議。

我們也會深入探討 AlloyDB 的 ScaNN 索引如何最佳化複雜查詢，以生成個人化風格建議。我們也會使用 Gemini 和 Imagen 這兩款強大的生成式 AI 模型，提供創意造型靈感，甚至可視需要生成個人專屬造型的圖片。這個應用程式完全採用無伺服器架構建構，確保使用者享有流暢且可擴充的體驗。

挑戰：這款應用程式提供個人化穿搭建議，協助難以決定如何穿搭的使用者。這也有助於避免因規劃服裝而感到決策疲勞。

解決方案：穿搭建議應用程式可為使用者提供智慧、個人化且引人入勝的時尚體驗，同時展示 AlloyDB、生成式 AI 和無伺服器技術的功能。

建構項目

本實驗室的學習內容包括：

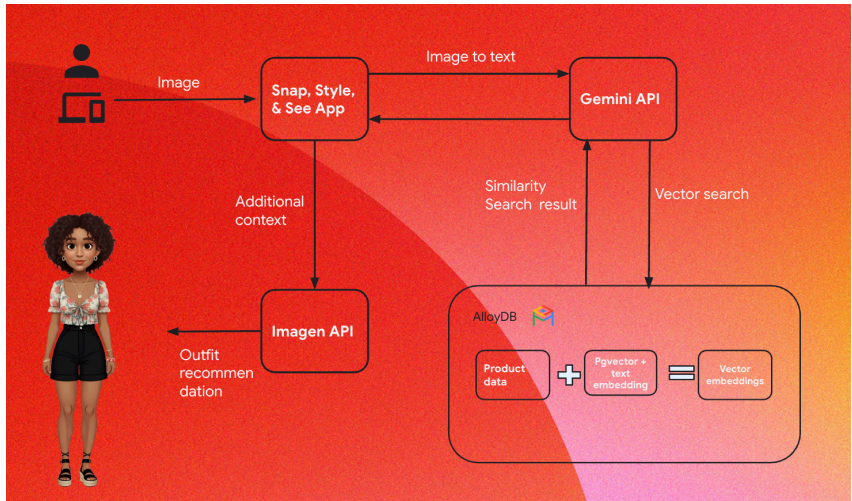
1. 建立 AlloyDB 執行個體並載入電子商務資料集
2. 在 AlloyDB 中啟用 pgvector 和生成式 AI 模型擴充功能
3. 根據產品說明生成嵌入
4. 在無伺服器 Cloud Run 函式中部署解決方案
5. 將圖片上傳至 Gemini，然後生成圖片說明提示。
6. 根據提示和電子商務資料集嵌入內容產生搜尋結果。
7. 新增其他提示來自訂提示，並生成樣式建議。
8. 在無伺服器 Cloud Run 函式中部署解決方案

需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
 - 已啟用計費功能的 Google Cloud 專案。
-

2. 架構

應用程式的高階架構如下：



以下各節將重點說明本教學課程的脈絡流程：

擷取：

首先，請將零售資料 (包括庫存、產品說明和顧客互動) 擷取至 AlloyDB。

Analytics Engine：

我們將使用 AlloyDB 做為分析引擎，執行下列作業：

1. **情境擷取：**引擎會分析 AlloyDB 中儲存的資料，瞭解產品、類別、消費者行為等之間的關係。
2. **建立嵌入：**系統會為使用者查詢和 AlloyDB 中儲存的資訊生成嵌入 (文字的數學表示)。
3. **向量搜尋：**引擎會執行相似度搜尋，比較查詢嵌入與產品說明、評論和其他相關資料的嵌入。這會找出 25 個最相關的「最鄰近項目」。

Gemini 建議：

圖片位元組陣列會透過 Vertex AI API 傳遞至 Gemini 模型，並附上提示，要求提供上衣的文字說明，以及下身穿搭建議。

AlloyDB RAG 和向量搜尋：

系統會使用上衣的說明查詢資料庫。查詢會將搜尋文字 (Gemini 模型建議的相符下半身服飾) 轉換為嵌入，並對資料庫中儲存的嵌入執行 Vector Search，找出最鄰近的項目 (相符結果)。AlloyDB 資料庫中的向量嵌入項目會使用 ScaNN 索引編入索引，以提升召回率。

生成回覆圖片：

經過驗證的回應會建構為 JSON 陣列，整個引擎則會封裝到無伺服器 Cloud Run 函式中，並從 Agent Builder 叫用。

Imagen 圖像生成：

系統會結合使用者的風格提示、使用者選取的建議和任何個人化要求，提示 Imagen 3 使用現有圖片。系統會根據這個提示詞，使用 Vertex AI API 生成造型圖片。

3. 事前準備

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境，預先載入了 bq。點按 Google Cloud 控制台頂端的「啟用 Cloud Shell」圖示



。

4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 執行下列指令，確認日後的 gcloud 指令會正確識別您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令明確設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 啟用必要的 API。

按照[連結](#)啟用 API。

如果忘記啟用任何 API，隨時可以在導入過程中啟用。

如要進一步瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

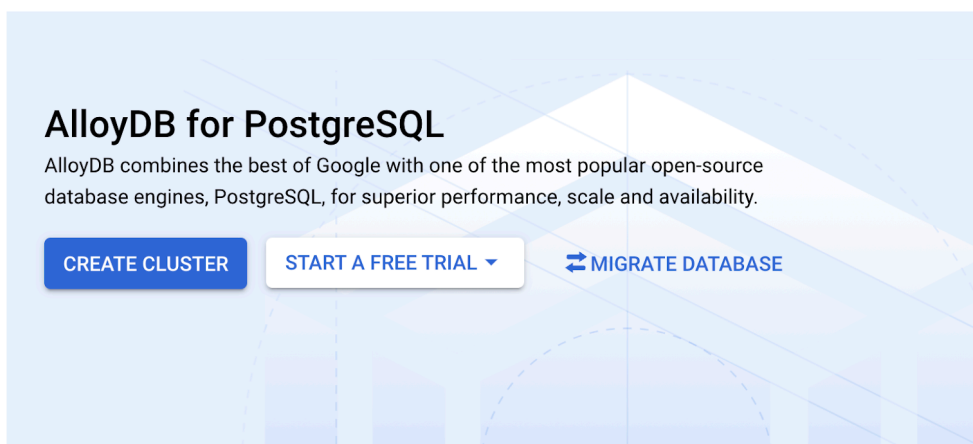
4. 資料庫設定

在本實驗室中，我們將使用 AlloyDB 做為資料庫，儲存零售電子商務資料集。並使用叢集儲存所有資源，例如資料庫和記錄檔。每個叢集都有一個「主要執行個體」，可做為資料的存取點。資料表是儲存資料的實際資源。

我們來建立 AlloyDB 叢集、執行個體和資料表，以便載入電子商務資料集。

建立叢集和執行個體

1. 在 Google Cloud 控制台中，搜尋「AlloyDB」 AlloyDB。如要在 Cloud 控制台尋找大部分的頁面，只要使用控制台的搜尋列搜尋即可。
2. 按一下 [建立叢集]。



3. 使用下列值建立叢集和執行個體：

- 叢集 ID：「shopping-cluster」
- 密碼：「alloydb」
- 與 PostgreSQL 15 相容
- 區域：「us-central1」
- 網路：「default」

Configure your cluster

Cluster ID *

Use lowercase letters, numbers, and hyphens. Start with a letter.

Password *



[GENERATE](#)

Set a password for the default "postgres" user. A password is required for the user to log in.

Database version *

PostgreSQL 15 compatible



Location

For better performance, keep your data close to the services that need it. Choice is permanent.

Region *

us-east4 (Northern Virginia)



Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *



[SHOW ADVANCED OPTIONS](#)

4. 在「網路」中選取預設網路時，會顯示下列選項。按一下「設定連線」即可設定預設網路。

Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *

default



Private services access connection required

Your network "default" requires a private services access connection. This connection enables your services to communicate exclusively by using internal IP addresses. [Learn more](#)

[SET UP CONNECTION](#)

5. 選取「使用系統自動分配的 IP 範圍」，然後按一下「繼續」。確認資訊後，按一下「建立連線」。

✓ Enable Service Networking API

2 Allocate an IP range

Google will use this allocated IP range to create subnets.

☐ Select one or more existing IP ranges or create a new one

Select or create an IP range

☒ Use an automatically allocated IP range

Google will automatically allocate an IP range of prefix-length /20 and use the name "default-ip-range".

CONTINUE

3 Create a connection

CREATE CONNECTION

CANCEL

等待預設網路建立完成。

6. 在「設定主要執行個體」中，將執行個體 ID 設為「shopping-instance」。

Configure your primary instance

Name your instance

Give your instance an ID.

Instance ID *
shopping-instance

Choice is permanent. Use lowercase letters, numbers, and hyphens. Start with a letter.

Zonal availability

☐ Single zone

In case of outage, no failover. Not recommended for production.

☒ Multiple zones (Highly available)

Automatic failover to another zone within your selected region. Recommended for production instances. Increases cost.

Machine

AlloyDB's superior performance lets you run your existing workloads on smaller machines.

Machine Type *
8 vCPU, 64 GB

Connectivity

Private IP Connectivity

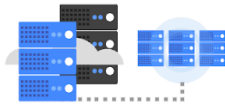
☒ Enable Private IP (always enabled)

Connect from applications and clients within the associated network. [Learn more](#)

7. 按一下「CREATE CLUSTER」(建立叢集)，然後完成叢集設定，步驟如下：

建立叢集約需 10 分鐘。成功後，「狀態」欄位會顯示「就緒」狀態。以下是顯示可供使用的叢集畫面範例。

Overview



Clusters are groups of instances of virtual machines that can include a primary instance and multiple read pool instances. All cluster resources share a storage layer, which scales as needed. [Learn more](#)

Status	Ready
Cluster ID	shopping-cluster
Version	PostgreSQL-15 compatible
Cluster type	Primary cluster
High availability	Highly available (multi-zone)
Location	us-central1 (Iowa) Low CO2
Total storage used	52.27 MB
Network	default
Issues	No issues found
Maintenance	Upcoming: No maintenance currently scheduled. EDIT
SHOW DETAILS	

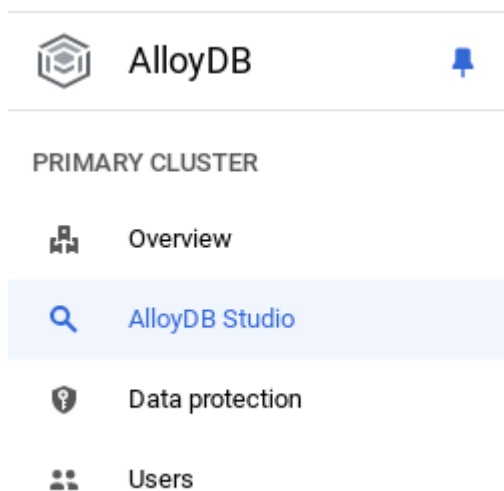
步驟 5 · 約 0 分鐘

5. 資料擷取

現在要新增包含商店資料的表格。等待執行個體建立完成。建立完成後，您可以使用建立叢集時設定的憑證登入 AlloyDB。

向 AlloyDB 資料庫驗證

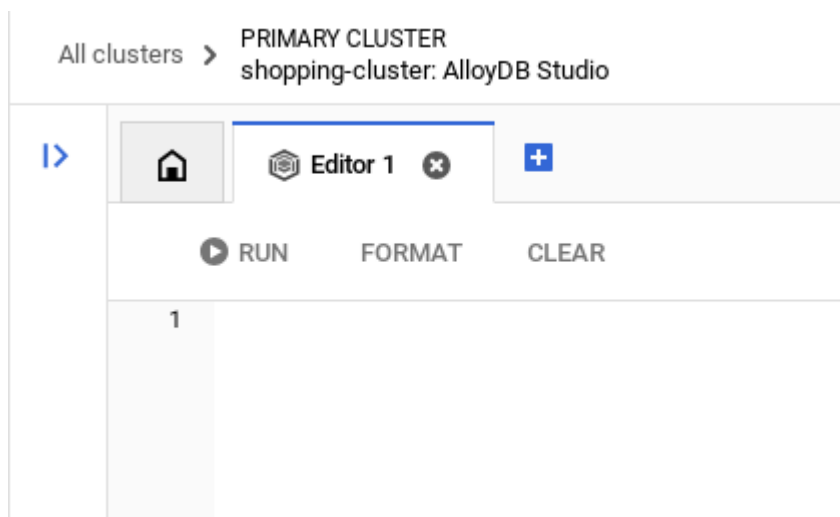
1. 在 Google Cloud 控制台中，前往「AlloyDB」[AlloyDB](#)。選取主要叢集，然後按一下左側導覽面板中的「AlloyDB Studio」：



2. 輸入下列詳細資料，向 AlloyDB 資料庫進行驗證：

- 使用者名稱：「postgres」
- 資料庫：「postgres」
- 密碼：「alloydb」

成功驗證 AlloyDB Studio 後，即可在「編輯器」分頁中輸入 SQL 指令。如要新增多個編輯器視窗，請按一下第一個編輯器分頁右側的加號。



您會在「編輯器」視窗中輸入 AlloyDB 指令，並視需要使用「執行」、「格式化」和「清除」選項。

啟用擴充功能

我們會使用「pgvector」和「google_ml_integration」擴充功能建構這個應用程式。

- [pgvector 擴充功能](#)可讓您儲存及搜尋向量嵌入。
- [google_ml_integration](#) 擴充功能提供您用來存取 Vertex AI 預測端點的函式，以便在 SQL 中取得預測結果。

1. 執行下列 DDL，[啟用](#)這些擴充功能：

```
CREATE EXTENSION IF NOT EXISTS google_ml_integration CASCADE;  
CREATE EXTENSION IF NOT EXISTS vector;
```

2. 執行下列 SQL 指令，確認擴充功能是否已安裝：

```
select extname, extversion from pg_extension;
```

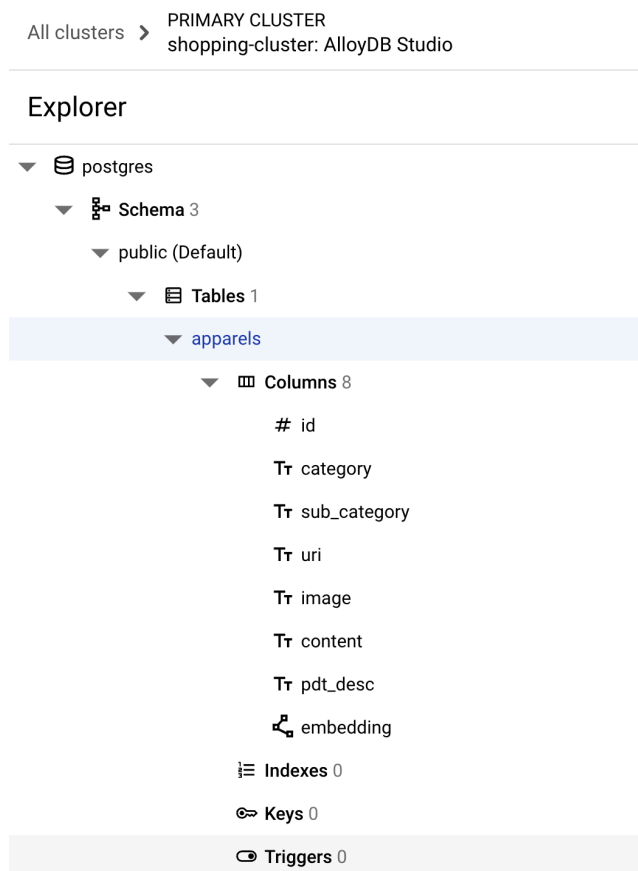
建立資料表

1. 使用下列 DDL 陳述式建立資料表：

```
CREATE TABLE  
  apparels ( id BIGINT,  
             category VARCHAR(100),  
             sub_category VARCHAR(50),  
             uri VARCHAR(200),  
             image VARCHAR(100),  
             content VARCHAR(2000),  
             pdt_desc VARCHAR(5000),  
             embedding vector(768) );
```

成功執行上述指令後，您應該就能在

◦ 請參閱下圖範例：



擷取資料

在本實驗室中，我們在 [SQL 檔案](#) 中提供約 200 筆記錄的測試資料。其中包含 `id`, `category`, `sub_category`, `uri`, `image` 和 `content`。其他欄位會在實驗室的後續部分填寫。

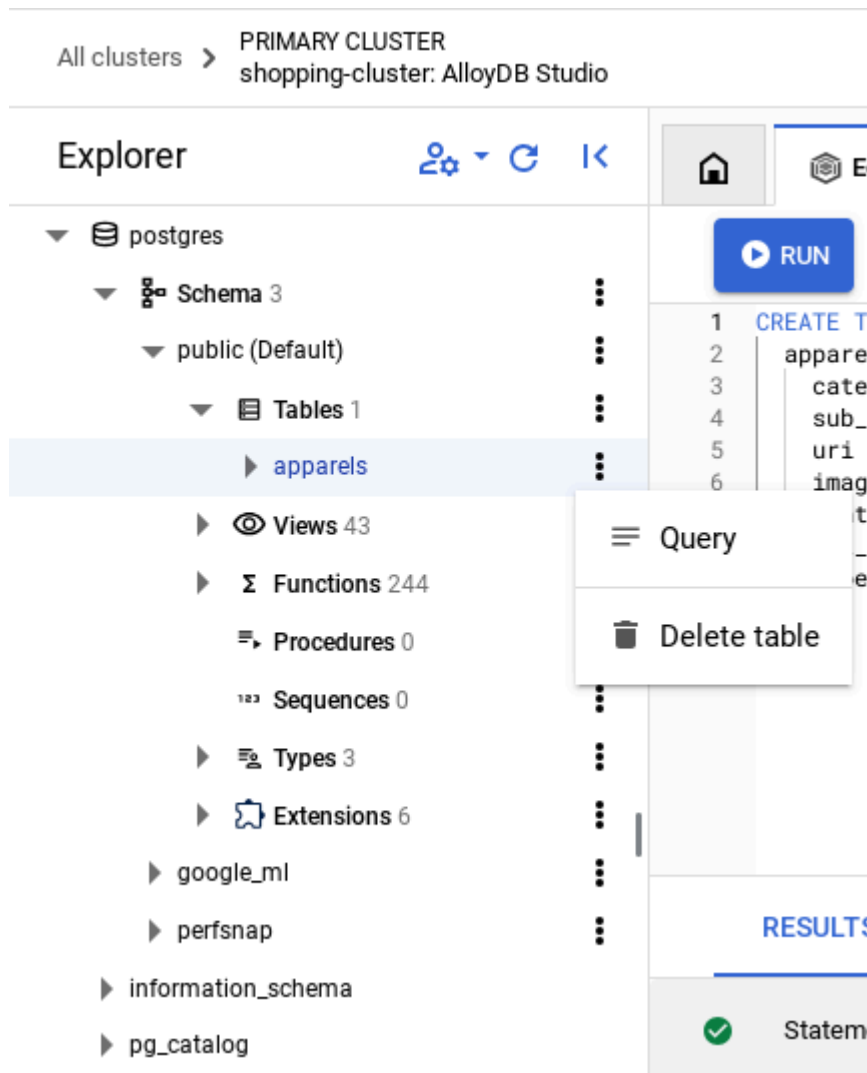
1. 在 AlloyDB Studio 的新編輯器分頁中，從 SQL 檔案複製 20 行/插入陳述式，然後按一下「RUN」(執行)**RUN**。

視您使用的 Google Cloud Billing 帳戶而定，資料列數量

可擷取的資料可能有所不同。如果遇到問題，請將列數減少至 10

或更低版本。

2. 展開「Explorer」部分，直到看到名為 `apparels` 的資料表。
3. 按一下選單圖示 [:]，然後按一下「查詢」。系統會在新編輯器分頁中開啟 SELECT 陳述式。



4. 按一下「執行」，確認已插入資料列。

授予使用者權限

我們將授予 `postgres` 使用者在 AlloyDB 中生成嵌入內容的權限。在 AlloyDB Studio 中執行下列陳述式，將 `embedding` 函式的執行權授予使用者 `postgres`：

```
GRANT EXECUTE ON FUNCTION embedding TO postgres;
```

為 AlloyDB 服務帳戶授予 Vertex AI 使用者角色

我們將使用 Vertex AI 的文字嵌入模型生成嵌入，並將 Vertex AI 使用者角色指派給 AlloyDB 服務帳戶。

在 Google Cloud 控制台中，按一下「Cloud Shell 終端機」[



] 圖示，然後執行下列指令：

```
PROJECT_ID=$(gcloud config get-value project)
```

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member="serviceAccount:service-$(gcloud projects describe $PROJECT_ID --  
format="value(projectNumber)")@gcp-sa-alloydb.iam.gserviceaccount.com" \  
  --role="roles/aiplatform.user"
```

6. 版本結構定義

如要建立嵌入內容，我們需要 `context`，也就是要納入單一欄位的所有資訊。我們會建立產品說明，並儲存在 `apparels` 資料表的 `pdt_desc` 欄中。

在本範例中，我們會使用各項產品的所有資訊，但您使用自己的資料時，可以隨意設計資料，只要對您的業務有意義即可。

在 AlloyDB Studio 的「編輯器」分頁中，執行下列指令，使用比對內容資料更新 `pdt_desc` 欄位：

```
UPDATE
  apparels
SET
  pdt_desc = CONCAT('This product category is: ', category, ' and sub_category is: ',
    sub_category, '. The description of the product is as follows: ', content, '. The product
    image is stored at: ', uri)
WHERE
  id IS NOT NULL;
```

這項 DML 會使用資料表中的所有可用欄位和其他依附元件 (如果您的用途有任何依附元件) 的資訊，建立簡單的內容摘要。如要更精確地分類資訊及建立內容，請隨意設計資料，找出對貴商家有意義的資料。

7. 為情境建立嵌入

電腦處理數字比處理文字容易得多。嵌入系統會將文字轉換為一系列浮點數，這些數字應代表文字，無論文字的措辭、使用的語言等為何。

例如描述海邊的地點。可能稱為「on the water」、「beachfront」、「walk from your room to the ocean」、「sur la mer」、「на берегу океана」等。這些字詞的外觀各不相同，但語意或機器學習術語 (即嵌入) 應非常接近。

資料和內容準備就緒後，我們將執行 SQL，將產品說明 (`pdt_desc`) 資料欄的嵌入內容新增至 `embedding` 欄位中的資料表。您可以使用各種嵌入模型。我們使用 Vertex AI 的 `text-embedding-005`。

請務必在整個專案中使用相同的嵌入模型。

1. 在 AlloyDB Studio 中執行下列指令，生成嵌入內容，並使用儲存資料的嵌入內容更新 `pdt_desc` 欄：

如果您使用先前建立的 Google Cloud 專案，可能需要繼續使用舊版文字嵌入模型，例如 `textembedding-gecko`。

```
UPDATE
  apparels
SET
  embedding = embedding( 'text-embedding-005',
    pdt_desc)
WHERE
  TRUE;
```

2. 執行下列指令，確認是否已產生嵌入：

```
SELECT
  id,
  category,
  sub_category,
  content,
  embedding
FROM
  Apparels
LIMIT 5;
```

以下是嵌入向量範例，看起來像是浮點數陣列，適用於查詢中的範例文字，如下所示：

```
select id, category, sub_category, content, embedding from apparels;
```

RESULTS

Apparel	Topwear	Girls Tops Pink Casual Palm Tree Girls Pink Top	
			[0.014596316,-0.03846882,-0.038603388,0.0060634837,0.03886954,0.0040010866,0.073628664,-0.01995821,-8.1448394e-05,0.054226466,0.07164986,-0.004241335,0.0019124502,0.0039123986,-0.0024608225,-0.045527022,0.017409893,0.031920653,0.04763973,-0.042894326,0.007723384,-0.006228419,0.016617483,-0.020080574,-0.0012002785,-0.008212429,0.014974915,-0.03253931,-0.02873162,-0.0030522568,-0.06529046,0.031061204,-0.018363003,0.029087069,-0.0038010685,-0.04053071,-0.0068419403,0.010498182,0.06123379,0.06135948,-0.017419688,0.020212132,-0.014141041,-0.025288673,0.007944778,-0.0031779595,-0.0034434984,0.00860807,0.00043757434,-0.059628054,-0.015224135,0.017263819,0.0466727,-0.04255475,0.009060793,-0.05668718,0.02130441,0.01461606,-0.02464654,-0.013048486,-0.03157506,-0.0077514537,-0.008257657,0.024717083,-0.012033532,-0.013686471,0.01648141,0.023388749,0.0761798,-0.023578823,-0.027780239,-0.038580347,0.044714082,-0.029784098,-0.0048513813,-0.070841506,-0.027332483,0.03883139,0.025170851,-0.014740545,-0.013489334,-0.008596852,0.0077242726,-0.03619594,-0.058110435,0.

在免費方案下建立的 Google Cloud 專案，可能會遇到配額問題，例如每秒允許的嵌入要求數。建議您使用 ID 的篩選查詢，然後在產生嵌入內容時，選擇性地選取 1 到 5 筆記錄等。

8. 執行向量搜尋

資料表、資料和嵌入都已準備就緒，現在就來對使用者搜尋文字執行即時向量搜尋。

假設使用者的搜尋文字為「pink color, casual, pure cotton tops for women」

如要找出與這項查詢相符的結果，請執行下列 SQL 查詢：

```
SELECT
id,
category,
sub_category,
content,
pdt_desc AS description
FROM
apparels
ORDER BY
embedding <=> embedding('text-embedding-005',
'pink color, casual, pure cotton tops for women')::vector
LIMIT
5;
```

讓我們詳細瞭解這項查詢：

在這項查詢中，

- 1. 使用者的搜尋文字為：「I want womens tops, pink casual only pure cotton.」
- 2. 我們會使用 `embedding()` 方法和模型：`text-embedding-005`，將這段搜尋文字轉換為嵌入。上一個步驟中，我們已將嵌入函式套用至表格中的所有項目，因此這個步驟應該很熟悉。
- 3. 「`<=>`」代表使用「餘弦相似度」距離方法。如要查看所有可用的相似度測量方式，請參閱 [pgvector 說明文件](#)。
- 4. 我們會將嵌入方法結果轉換為 **vector** 資料型別，以便與資料庫中儲存的向量相容。
- 5. **LIMIT 5** 代表我們要為搜尋文字擷取 5 個最鄰近的項目。

以下顯示這項 SQL 查詢的回應範例：

RESULTS

id	category	sub_category	content	description	
63	Apparel	Topwear	Girls Tops White Casual Gini Jony Girls Printed White Top	This product category is: Apparel and sub_category is: Topwear. The d...	▼
56	Apparel	Topwear	Girls Tops Pink Casual Gini and Jony Girls Pink Top	This product category is: Apparel and sub_category is: Topwear. The d...	▼
57	Apparel	Topwear	Girls Tops Pink Casual Gini and Jony Girls Pretty Pink Top	This product category is: Apparel and sub_category is: Topwear. The d...	▼
94	Apparel	Topwear	Girls Tops White Casual Gini and Jony Girls Printed Whit...	This product category is: Apparel and sub_category is: Topwear. The d...	▼
12	Apparel	Topwear	Girls Tops Pink Casual Gini and Jony Girls Pretty Blossom...	This product category is: Apparel and sub_category is: Topwear. The d...	▼

如果您在結果中看到的，相符項目與搜尋文字相當接近。請嘗試變更顏色，看看結果會如何變化。

AlloyDB ScaNN 索引，提升查詢效能

假設我們現在想使用 **ScaNN** 索引，提升這項向量搜尋結果的效能 (查詢時間)、效率和召回率。

請參閱這篇網誌中的步驟，比較有索引和沒有索引的結果差異。

如要使用 ScaNN 索引，請嘗試下列步驟：

1. 由於我們已建立叢集、執行個體、內容和嵌入，因此只需使用下列陳述式安裝 ScaNN 擴充功能：

```
CREATE EXTENSION IF NOT EXISTS alloydb_scann;
```

2. 建立 ScaNN 索引：

```
CREATE INDEX apparel_index ON apparels
USING scann (embedding cosine)
WITH (num_leaves=54);
```

在上述 DDL 中：

- `apparel_index` 是索引的名稱。
- `apparels` 是資料表名稱。
- `scann` 是索引方法。
- `embedding` 是您要建立索引的資料表欄。
- `cosine` 是您要用於索引的距離方法。
- `54` 是要套用至這個索引的分區數量。請設為 1 到 1048576 之間的任何值。如要進一步瞭解如何決定這個值，請參閱「[調整 ScaNN 索引](#)」。

根據 [ScaNN repo](#) 的建議，我們使用資料點數的平方根。分區時，`num_leaves` 大致應為資料點數量的平方根。

3. 使用下列查詢檢查索引是否已建立：

```
SELECT * FROM pg_stat_ann_indexes;
```

4. 使用與不含索引時相同的查詢，執行向量搜尋：

```
select * from apparels
  ORDER BY embedding <=> CAST(embedding('textembedding-gecko', 'white tops for girls without
any print') as vector(768))
  LIMIT 20
```

上述查詢與實驗室步驟 8 中使用的查詢相同。不過，現在我們已使用 ScaNN 索引為該欄位建立索引。

5. 使用索引和不使用索引，分別執行簡單的搜尋查詢進行測試。如要測試不含索引的查詢，請捨棄索引：

```
white tops for girls without any print
```

在 INDEXED 嵌入資料中，使用上述搜尋文字查詢 Vector Search，可獲得優質的搜尋結果並提升效率。使用索引後，效率大幅提升 (執行時間：**不使用 ScaNN** 時為 10.37 毫秒，**使用 ScaNN** 時為 0.87 毫秒)。如要進一步瞭解這個主題，請參閱這篇[網誌](#)。

9. 使用 LLM 驗證比對結果

在繼續操作並建立服務，以便向應用程式傳回最佳相符項目之前，請先使用生成式 AI 模型驗證這些潛在回覆是否確實相關，且可安全地與使用者分享。

確認執行個體已設定為 Gemini

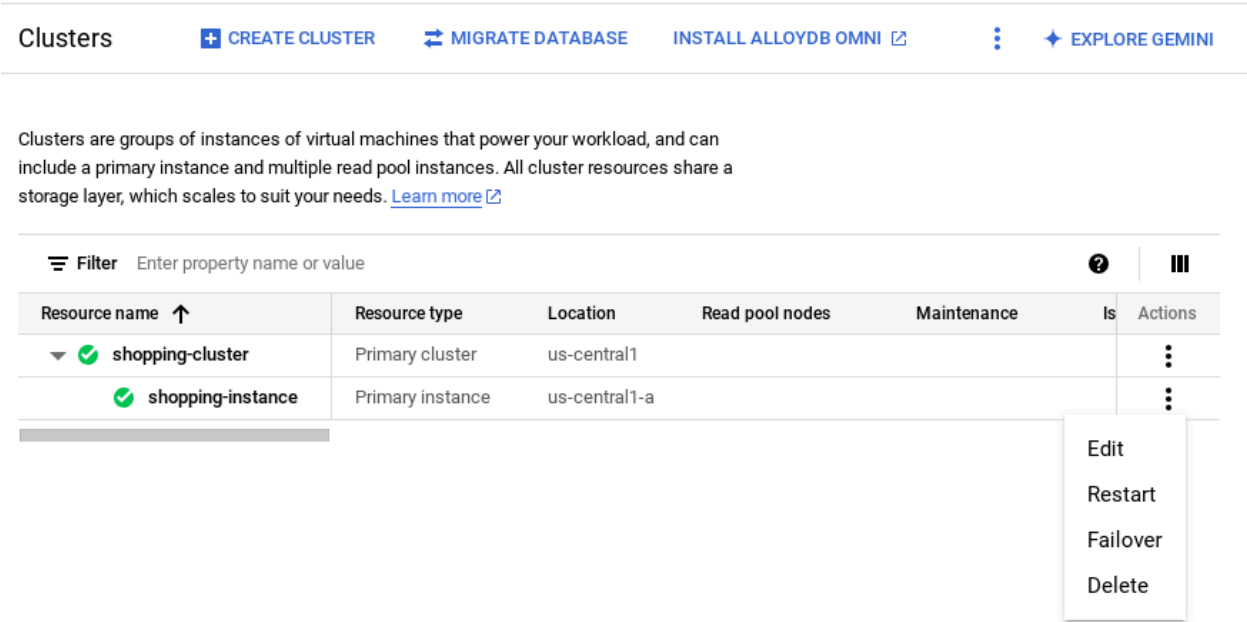
1. 確認叢集和執行個體已啟用 `google_ml_integration`。在 AlloyDB Studio 中執行下列指令：

```
show google_ml_integration.enable_model_support;
```

如果值顯示為「on」，請略過接下來的 2 個步驟，直接設定

AlloyDB 和 Vertex AI 模型整合。

1. 前往 AlloyDB 叢集的主要執行個體，然後按一下「編輯主要執行個體」。



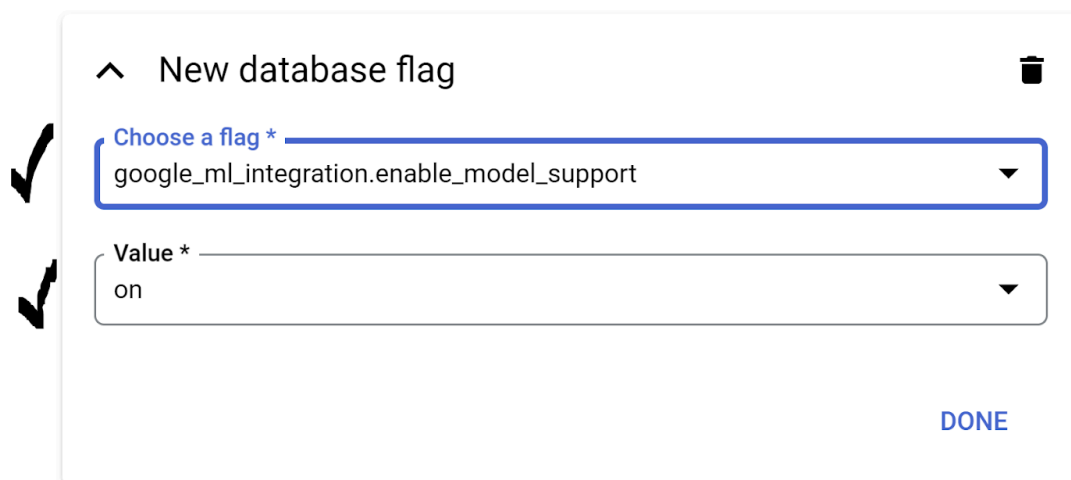
2. 在 進階設定選項 中，展開「新增 資料庫旗標」部分，並確認 `google_ml_integration.enable_model_support flag` 已設為「on」，如下所示：

← Edit primary instance

Flags

Use flags to customise your instance, if needed. [Learn more](#)

Enable the columnar engine flag to improve performance and query speed of HTAP and OLAP workloads. [Learn more](#)



^ New database flag

Choose a flag *
google_ml_integration.enable_model_support

Value *
on

DONE

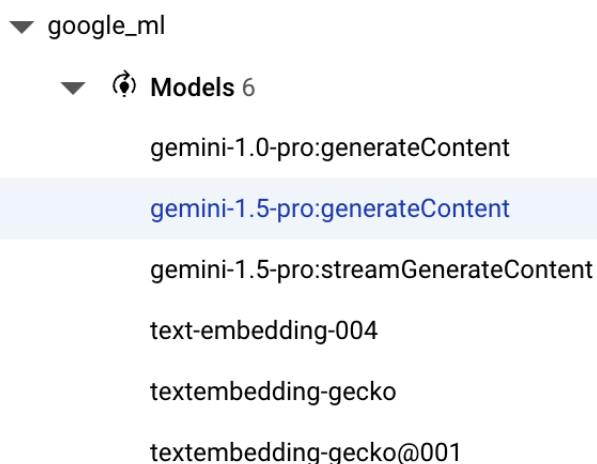
3. 如果未設為「on」，請設為「on」，然後按一下「UPDATE INSTANCE」(更新執行個體)。

這個步驟需要幾分鐘才能完成。

整合 AlloyDB 和 Vertex AI 模型

現在您可以連線至 AlloyDB Studio，並執行下列 DML 陳述式，從 AlloyDB 設定 Gemini 模型存取權 (請使用標示位置的專案 ID)。執行指令前，系統可能會警告您有語法錯誤，但指令應該可以正常執行。

1. 在 Google Cloud 控制台中，前往「AlloyDB」[AlloyDB](#)。選取主要叢集，然後按一下左側導覽面板中的「AlloyDB Studio」。
2. 我們會使用 `google_ml_integration` 擴充功能預設提供的 `gemini-1.5-pro:generateContent`。



3. 您可以在 AlloyDB Studio 中執行下列指令，查看設定可存取的模型：

```
select model_id,model_type from google_ml.model_info_view;
```

4. 執行下列指令，授予資料庫使用者執行 `ml_predict_row` 函式的權限，以便使用 Google Vertex AI 模型執行預測：

```
GRANT EXECUTE ON FUNCTION ml_predict_row to postgres;
```

如果您使用現有的 Google Cloud 專案，以及先前建立的 AlloyDB 叢集/執行個體，可以執行 `ALTER EXTENSION google_ml_integration UPDATE;` 指令，將 `google_ml_integration` 版本升級至最新版本，存取內建支援的模型。

評估回覆

雖然我們會在下一節中實際將應用程式移至 Cloud Run，並使用一個大型查詢，但為了確保查詢的回應合理，查詢可能難以理解。

我們會查看各個區段，這些區段會建構出我們最終使用的較大查詢。

1. 首先，我們會向資料庫傳送要求，取得與使用者查詢內容**最接近的 5 個結果**。為了簡化作業，我們將查詢硬式編碼，但請放心，稍後我們會將查詢內插至查詢中。

我們將納入 `apparels` 資料表中的產品說明，並新增兩個欄位，一個是說明與索引的組合，另一個則是原始要求。這項資料會儲存在名為 `xyz` 的資料表中，這是暫時性資料表名稱。

```
CREATE TABLE
  xyz AS
SELECT
  id || ' - ' || pdt_desc AS literature,
  pdt_desc AS content,
  'I want womens tops, pink casual only pure cotton.' AS user_text
FROM
  apparels
ORDER BY
  embedding <=> embedding('text-embedding-005',
    'I want womens tops, pink casual only pure cotton.')::vector
LIMIT
  5;
```

這項查詢的輸出內容會是與使用者查詢相關的**5 個最相似的資料列**。The

新資料表 `xyz` 會包含 5 個資料列，每個資料列都會有下列資料欄：

- `literature`
- `content`
- `user_text`

2. 為判斷回覆的有效性，我們會使用複雜的查詢，說明如何評估回覆。並在 `xyz` 資料表中，將 `user_text` 和 `content` 用於查詢。

```
"Read this user search text: ', user_text,
' Compare it against the product inventory data set: ', content,
' Return a response with 3 values: 1) MATCH: if the 2 contexts are at least 85% matching or
not: YES or NO 2) PERCENTAGE: percentage of match, make sure that this percentage is accurate
3) DIFFERENCE: A clear short easy description of the difference between the 2 products.
Remember if the user search text says that some attribute should not be there, and the record
has it, it should be a NO match."
```

3. 然後使用該查詢，檢查 `xyz` 資料表中的回應「品質」。所謂的「優良程度」，是指生成的回覆與我們預期的相符程度。

```
CREATE TABLE
  x AS
SELECT
  json_array_elements( google_ml.predict_row( model_id => 'gemini-1.5',
    request_body => CONCAT('{
"contents": [
  { "role": "user",
    "parts":
      [ { "text": "Read this user search text: ', user_text, ' Compare it against the
product inventory data set: ', content, ' Return a response with 3 values: 1) MATCH: if the 2
contexts are at least 85% matching or not: YES or NO 2) PERCENTAGE: percentage of match, make
sure that this percentage is accurate 3) DIFFERENCE: A clear short easy description of the
difference between the 2 products. Remember if the user search text says that some attribute
should not be there, and the record has it, it should be a NO match.'"
      } ]
    } ]
  } )::json))-> 'candidates' -> 0 -> 'content' -> 'parts' -> 0 -> 'text'
AS LLM_RESPONSE
FROM
  xyz;
```

4. `predict_row` 會以 JSON 格式傳回結果。程式碼「`-> 'candidates' -> 0 -> 'content' -> 'parts' -> 0 -> 'text'`」用於從該 JSON 擷取實際文字。如要查看實際傳回的 JSON，可以移除這段程式碼。

5. 最後，如要取得 LLM 欄位，只要從 `x` 資料表擷取即可：

```
SELECT
  LLM_RESPONSE
FROM
  x;
```

6. 這可以合併為單一查詢，如下所示：

警告：如果您已執行上述查詢來檢查中繼結果，

請務必先從 AlloyDB 資料庫中刪除/移除 `xyz` 和 `x` 資料表，再執行這項查詢。

```
SELECT
  LLM_RESPONSE
FROM (
  SELECT
    json_array_elements( google_ml.predict_row( model_id => 'gemini-1.5',
      request_body => CONCAT('{
        "contents": [
          { "role": "user",
            "parts":
              [ { "text": "Read this user search text: ', user_text, ' Compare it against the
product inventory data set: ', content, ' Return a response with 3 values: 1) MATCH: if the 2
contexts are at least 85% matching or not: YES or NO 2) PERCENTAGE: percentage of match, make
sure that this percentage is accurate 3) DIFFERENCE: A clear short easy description of the
difference between the 2 products. Remember if the user search text says that some attribute
should not be there, and the record has it, it should be a NO match."
              } ]
            }
          ] }'
      )::json)) -> 'candidates' -> 0 -> 'content' -> 'parts' -> 0 -> 'text'
  AS LLM_RESPONSE
  FROM (
    SELECT
      id || ' - ' || pdt_desc AS literature,
      pdt_desc AS content,
      'I want womens tops, pink casual only pure cotton.' user_text
    FROM
      apparels
    ORDER BY
      embedding <=> embedding('text-embedding-005',
        'I want womens tops, pink casual only pure cotton.')::vector
    LIMIT
      5 ) AS xyz ) AS X;
```

較大的查詢是我們在先前步驟中執行的所有查詢的組合。結果會顯示是否有相符的內容、相符的百分比，以及評分の説明。

請注意，Gemini 模型預設為「開啟」串流，因此實際回覆會分散在多行：

RESULTS
llm_response
"Here"
"s a breakdown of the comparison:\n\n**1) MATCH: NO**"
"\n\n**2) PERCENTAGE: 65%**\n\n**3)"
") DIFFERENCE:**\n\n**Target Audience:** The user explicitly wants \"womens tops\", while the product is a girls' top. This is a significant mismatch"
".\n\n**Fabric:** The user desires \"pure cotton\" but the product description doesn't mention the fabric composition.\n\n**Style Detail:** "
"The product description includes \"Doodle\" and \"Retro\" which are not mentioned in the user's search. \n\n**Explanation of Percentage:** While \"pink\", \"casual\", and \"tops\" align, the core mismatches in"
" demographics and missing fabric information make it less than an 85% match. \n"
"

10. 將應用程式帶到網頁

現在我們要代管這個應用程式，讓使用者可以透過網際網路存取。

建立 Cloud Run 函式

1. 在 Google Cloud 控制台中，使用下列連結前往 Cloud Run Functions：

<https://console.cloud.google.com/run/create?deploymentType=function>

2. 在「設定」中，將函式名稱設為「retail-engine」，並選取「us-central1」做為區域。
3. 在「Endpoint URL」中，選取執行階段「Java 17」。
4. 在「驗證」中，選取「允許未經驗證的叫用」。
5. 展開「Container(s), Volumes, Networking, Security」，然後按一下「Networking」分頁標籤。
6. 選取 連線至虛擬私有雲以傳出流量，然後按一下 使用無伺服器 VPC 存取連接器。
7. 在「網路」中，按一下「新增虛擬私有雲連接器」。如果尚未啟用，請啟用 無伺服器 VPC 存取 API。
8. 在「建立連接器」中，將名稱設為 `alloydb-test-conn`。
9. 將區域設為 `us-central`。
10. 將「網路」值保留為「預設」，並將「子網路」設為「自訂 IP 範圍」，IP 範圍為「10.8.0.0」或類似的可用範圍。
11. 展開顯示資源調度設定，然後將執行個體數量下限設為 2，並將執行個體數量上限設為 3。
12. 將執行個體類型選為 `f1-micro`。「建立連接器」選項如下所示：

Create connector

Name *

alloydb-test-conn

Region *

us-central1

A region is a specific geographical location where you can run your resources.

Network *

default

Subnet *

Custom IP range

Select an unused /28 subnet or create a new one by entering an unused /28 IP range. The VPC Connector will create connector instances on this subnet.

IP range *

10.8.0.0 /28

IP range must be an unused /28 CIDR range in your VPC network, such as 10.8.0.0/28. The VPC Connector will create connector instances on IP addresses in this range. Ensure the range does not overlap with an existing subnet. [Learn more](#)

Scaling Settings

Minimum instances *

2

The minimum number of instances provisioned at any time. The connector will automatically scale out if more capacity is needed. Minimum number of instances cannot be decreased later. Larger values increase your cost.

Maximum instances *

3

The maximum number of instances provisioned at any time. The connector will not automatically scale out above this value. Maximum number of instances cannot be decreased later. This setting limits your maximum cost.

 Connectors don't scale down automatically or manually. Once the connector has reached maximum number of instances it will remain at this number.

Instance type *

f1-micro

Larger instances support higher bandwidth but raise your costs. [Learn more](#)

[HIDE SCALING SETTINGS](#)

CREATE

CANCEL

13. 按一下「建立」即可建立連接器。
14. 在「流量路由」中，選取「將所有流量轉送至虛擬私有雲」。
15. 按一下「建立」即可建立函式。

部署應用程式

建立函式後，請更新來源並重新部署應用程式。

1. 在 [Cloud Run](#) 中，按一下「Services」分頁，然後點選「retail-engine」函式。
2. 按一下「來源」分頁標籤。將預設的「函式進入點」設為「`gcfv2.HelloHttpFunction`」。
3. 將 `HelloHttpFunction.java` 檔案的內容，替換成這個 [Java 檔案](#) 的內容。
4. 根據 AlloyDB 執行個體和叢集的詳細資料，更新檔案中的 `AlloyDbJdbcConnector` 詳細資料。將 `$PROJECT_ID` 替換為 AlloyDB 叢集和執行個體的專案 ID。

```
public HikariDataSource AlloyDbJdbcConnector() {
    HikariDataSource dataSource;
    String ALLOYDB_DB = "postgres";
    String ALLOYDB_USER = "postgres";
    String ALLOYDB_PASS = "alloydb";
    String ALLOYDB_INSTANCE_NAME = "projects/$PROJECT_ID/locations/us-central1/clusters/shopping-cluster/instances/shopping-instance";
    HikariConfig config = new HikariConfig();
    config.setJdbcUrl(String.format("jdbc:postgresql://%s", ALLOYDB_DB));
    config.setUsername(ALLOYDB_USER); // e.g., "postgres"
    config.setPassword(ALLOYDB_PASS); // e.g., "secret-password"
    config.addDataSourceProperty("socketFactory", "com.google.cloud.alloydb.SocketFactory");
    config.addDataSourceProperty("alloydbInstanceName", ALLOYDB_INSTANCE_NAME);
    dataSource = new HikariDataSource(config);
    return dataSource;
}
```

5. 將 `pom.xml` 檔案的內容換成這個 [XML 檔案](#) 的內容。
 6. 點選「儲存並重新部署」來部署函式。
-

11. 測試 retail-engine 應用程式

部署更新後的 Cloud 函式後，您應該會看到下列格式的端點：

如要找出 Google Cloud 雲端專案的專案編號，請前往 Google Cloud 控制台，依序點選左側導覽窗格中的「Cloud 總覽」>「資訊主頁」。您可以在「專案資訊」中找到專案編號。

```
https://retail-engine-PROJECT_NUMBER.us-central1.run.app
```

您可以在 Cloud Shell 終端機中執行下列指令進行測試：

```
gcloud functions call retail-engine --region=us-central1 --gen2 --data '{"search": "I want some kids clothes themed on Disney"}'
```

或者，您也可以按照下列步驟測試 Cloud Run 函式：

```
PROJECT_ID=$(gcloud config get-value project)

curl -X POST https://retail-engine-$PROJECT_NUMBER.us-central1.run.app \
  -H 'Content-Type: application/json' \
  -d '{"search": "I want some kids clothes themed on Disney"}' \
  | jq .
```

結果如下：

```
{
  {
    "id": "164",
    "category": "Apparel",
    "sub_category": "Topwear",
    "uri": "http://assets.myntassets.com/v1/images/style/properties/cf8defc53a6242dc720dc826c169ca11_images.jpg",
    "description": "This product category is: Apparel and sub_category is: Topwear. The description of the product is as follows: Girls Tops Pink Casual Disney Kids",
    "literature": "\"'s a breakdown of the comparison:\\n\\n**1) MATCH:** YES\\n\\n\""
  },
  {
    "id": "191",
    "category": "Apparel",
    "sub_category": "Topwear",
    "uri": "http://assets.myntassets.com/v1/images/style/properties/f68af9cde009efeb220efe1dd197a7f4_images.jpg",
    "description": "This product category is: Apparel and sub_category is: Topwear. The description of the product is as follows: Girls Tops Blue Casual Disney Kids",
    "literature": "\"**1) MATCH: YES**\\n\\n**2) PERCENTAGE: 9\\n\""
  },
  {
    "id": "100",
    "category": "Apparel",
    "sub_category": "Topwear",
    "uri": "http://assets.myntassets.com/v1/images/style/properties/d2c388bcd6a795e0d0cea56392d352a6_images.jpg",
    "description": "This product category is: Apparel and sub_category is: Topwear. The description of the product is as follows: Girls Tops White Casual Disney Kids",
    "literature": "\" MATCH:** YES\\n\\n**2) PERCENTAGE:** 90% \\n\\n\""
  },
  {
    "id": "119",
    "category": "Apparel",
    "sub_category": "Topwear",
    "uri": "http://assets.myntassets.com/v1/images/style/properties/9dfcd501502405f6befacbefbc8bec12_images.jpg",
    "description": "This product category is: Apparel and sub_category is: Topwear. The description of the product is as follows: Girls Tops Pink Casual Disney Kids",
    "literature": "\"'s a breakdown of the comparison:\\n\\n**1) MATCH:** YES\\n\\n\""
  },
  {
    "id": "130",
    "category": "Apparel",
    "sub_category": "Topwear",
    "uri": "http://assets.myntassets.com/v1/images/style/properties/35e3b397144b0e2f810be458cea913bc_images.jpg",
    "description": "This product category is: Apparel and sub_category is: Topwear. The description of the product is as follows: Girls Tops Red Casual Disney Kids",
    "literature": "\" MATCH:** YES\\n\\n**2) PERCENTAGE:** 90% \\n\\n\""
  }
}
```

我們已在 AlloyDB 資料上使用嵌入模型執行相似度向量搜尋，現在可以開始建立應用程式，使用這些嵌入和圖片及提示來生成樣式建議

12. 瞭解服裝建議流程

穿搭推薦應用程式是經過設定的 Sprint Boot 應用程式，可搭配我們在 AlloyDB 零售引擎應用程式中建立的嵌入內容，以及 Gemini 和 Imagen，生成穿搭造型選項。你也可以新增自訂提示，並即興調整建議。

舉例來說，你將衣櫃中一件桃紅色上衣的圖片上傳至這個應用程式。當你點選「顯示」時，應用程式會根據應用程式程式碼中設定的提示和 AlloyDB 資料庫中的嵌入內容，生成多個與原始圖片相符的選項。現在您想知道藍色項鍊的建議選項會是什麼樣子，因此在這些行中新增提示，然後按一下「樣式」。系統會結合原始圖片和建議，生成最終圖片，打造出相符的服裝。

如要開始建立服裝推薦應用程式，請按照下列步驟操作：

1. 在 [Cloud Run](#) 中開啟 **retail-engine** 應用程式，並記下應用程式的網址。這是用來產生類似建議的嵌入項目存放區。
2. 在 IDE 中，複製 <https://github.com/AbiramiSukumaran/outfit-recommender/> 存放區。在本練習中，顯示的步驟是在 Visual Studio Code IDE 中執行。

```
git clone https://github.com/AbiramiSukumaran/outfit-recommender/
```

以下是應用程式目錄中的一些重要檔案：

- `src/main`：應用程式檔案和 HTML 所在的來源目錄：
- `HelloWorldApplication.java`：Spring Boot 應用程式的主要進入點。
- `HelloWorldController.java`：Spring Boot REST 控制器，可處理與服裝推薦應用程式相關的 HTTP 要求。這個檔案會處理 GET 和 POST 要求、處理使用者提示、分析圖片、與 AlloyDB 嵌入互動，並將最終回應傳回使用者介面。這個控制器會呼叫 `GenerateImageSample` 類別。
- `GenerateImageSample.java`：包含圖像生成類別，可連線至 Vertex AI、設定使用者提示詞格式、對 Imagen 模型發出 API 呼叫，並將預測圖片傳回控制器類別。
- `Resources`：這個目錄包含產生應用程式 UI 時所需的圖片和 HTML 檔案。
- `Pom.xml`：定義專案依附元件和設定。

3. 在 Visual Studio Code 中開啟 `HelloWorldController.java`，然後根據 AlloyDB 執行個體的建立位置，更新專案 ID 和位置的執行個體。

```
/** Defines a controller to handle HTTP requests */
@RestController
public class HelloWorldController {

    private static final Logger logger = Logger.getLogger(HelloWorldController.class.getName());
    public static final String PROJECT_ID = "outfit-recommender-451707";
    public static final String LOCATION = "us-central1";

    /**
     * Create an endpoint for the landing page
     */
}
```

4. 將 `endpoint` 更新為您先前代管的 **retail-engine** 應用程式網址。

```

public ArrayList<ArrayList<String>> databaseRecommendationEngine(String searchText) {
    ArrayList<ArrayList<String>> res = new ArrayList<ArrayList<String>>();
    ArrayList<String> resuri = new ArrayList<String>();
    ArrayList<String> resmatch = new ArrayList<String>();
    String result = "";
    String endpoint = "https://retail-engine-232777358440.us-central1.run.app";
    //code to invoke the endpoint and pass the string request to retrieve Gemini val
    System.out.println(x:"Inside calling endpoint function*****");
    try {
        URL url = new URL(endpoint);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    }
}

```

5. 開啟 `GenerateImageSample.java`，然後根據 AlloyDB 執行個體的建立位置更新專案 ID 和位置。

```

public class GenerateImageSample {
    static String bytesBase64Encoded = "";
    static String bytesBase64EncodedOutput = "";
    Run | Debug
    public static void main(String[] args) throws IOException {
        String projectId = "outfit-recommender-451707";
        String location = "us-central1";
        String prompt = "Generate a cartoon performing the following Yoga pose: Malasana, kn
        // generateImage(projectId, location, prompt);
    }
}

```

如果是免費費率層級的專案，或是無法生成含有人物的圖片

圖，你可以修改 `post_context` 提示詞，`Generate a high-quality, aesthetically pleasing image suitable for fashion recommendations. Show a two piece outfit from the provided image without a human figure and using other details described in the prompt.` 產生風格建議。此外，請註解 `paramsMap.put("personGeneration", "allow_adult");` 設定。

6. 儲存所有檔案。

現在我們要將這個應用程式部署到 Cloud Run 無伺服器執行階段。

13. 將應用程式帶到網路上

我們已將相關專案、位置和零售引擎應用程式詳細資料新增至服裝建議工具 Spring Boot 應用程式，現在可以將應用程式部署至 Cloud Run。

我們將在 Visual Code Studio 終端機中使用 `gcloud run deploy` 指令部署應用程式。如果是 Visual Studio Code，您可以安裝 Google Cloud Code 擴充功能，開始使用 gcloud CLI。

如要部署應用程式，請按照下列步驟操作：

1. 在 IDE 中開啟複製的目錄，然後啟動終端機。如果是 Visual Code Studio，請依序點選「Terminal」>「New Terminal」。
2. 請按照[這份文件](#)中的操作說明安裝 gcloud CLI。
3. 如果您使用 Visual Code Studio，請按一下「Extensions」(擴充功能)，搜尋「Google Cloud Code」並安裝擴充功能。
4. 在 IDE 終端機中執行下列指令，驗證您的 Google 帳戶：

```
gcloud auth application-default login
```

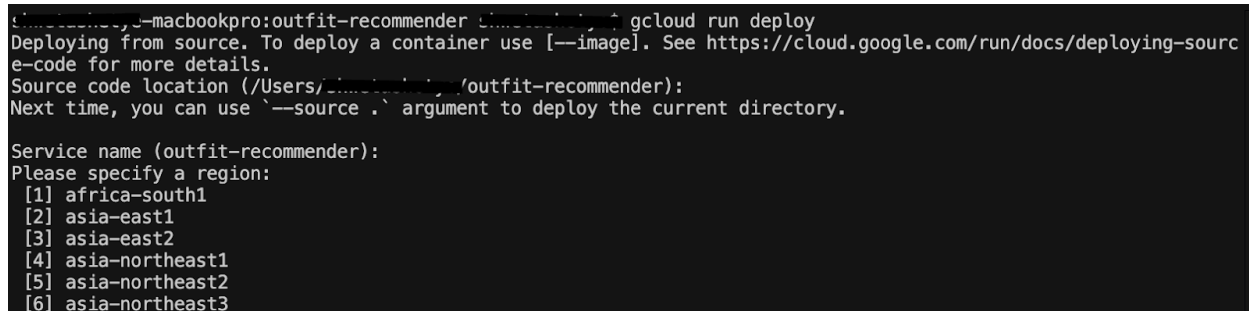
5. 將專案 ID 設為 AlloyDB 執行個體所在的專案。

```
gcloud config set project PROJECT_ID
```

6. 啟動部署程序。

```
gcloud run deploy
```

7. 在 `Source code location` 中，按 Enter 鍵選取複製的 GitHub 目錄。
8. 在 `Service name` 中輸入服務名稱，例如 `outfit-recommender`，然後按下 Enter 鍵。
9. 在 `Please specify a region` 中，輸入 AlloyDB 執行個體和 retail-engine 應用程式的代管位置，例如 `us-central1` 的代碼為 32，然後按下 Enter 鍵。



```
timothys-macbookpro:outfit-recommender timothy$ gcloud run deploy
Deploying from source. To deploy a container use [--image]. See https://cloud.google.com/run/docs/deploying-source-code for more details.
Source code location (/Users/timothys-macbookpro/outfit-recommender):
Next time, you can use '--source .' argument to deploy the current directory.

Service name (outfit-recommender):
Please specify a region:
[1] africa-south1
[2] asia-east1
[3] asia-east2
[4] asia-northeast1
[5] asia-northeast2
[6] asia-northeast3
```

10. 在 `Allow unauthenticated invocations to [..]` 中輸入 **Y**，然後按下 Enter 鍵。

下圖顯示應用程式的部署進度：

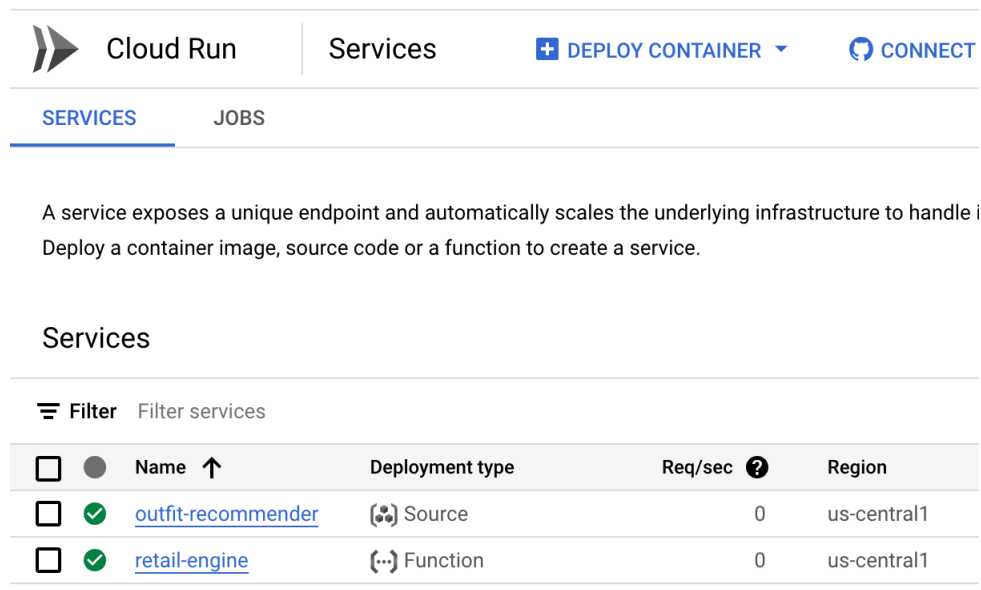
```
Building using Dockerfile and deploying container to Cloud Run service [outfit-recommender] in project [alloyndpoint] region [us-central1]
: Building and deploying new service... Uploading sources.
: Uploading sources...
. Building Container...
. Creating Revision...
. Routing traffic...
. Setting IAM Policy...
Creating temporary archive of 37 file(s) totalling 720.6 KiB before compression.
Some files were not included in the source upload.

Check the gcloud log [/Users/shwetashetye/.config/gcloud/logs/2025.02.27/19.44.28.766989.log] to see which fi
X Building and deploying new service... Done.
✓ Uploading sources...
✓ Building Container... Logs are available at [https://console.cloud.google.com/cloud-build/builds;region=u
entral1/4dae75a7-72f2-4ce2-804c-16ce4560b08c?project=22905290964].
✓ Creating Revision...
✓ Routing traffic...
. Setting IAM Policy...
Completed with warnings:
. Setting IAM policy failed, try "gcloud beta run services add-iam-policy-binding --region=us-central1 --memb
llUsers --role=roles/run.invoker outfit-recommender"
Service [outfit-recommender] revision [outfit-recommender-00001-mfc] has been deployed and is serving 100 per
of traffic.
Service URL: https://outfit-recommender-22905290964.us-central1.run.app
```

14. 測試服裝推薦應用程式

應用程式成功部署至 Cloud Run 後，您可以在 Google Cloud 控制台中看到服務，如下所示：

1. 前往 Google Cloud 控制台的「Cloud Run」。
2. 在「Services」中，按一下您部署的「outfit recommender」服務。您應該會看到 **retail-engine** 和 **outfit-recommender** 服務，如下所示：



Cloud Run Services

DEPLOY CONTAINER CONNECT

SERVICES JOBS

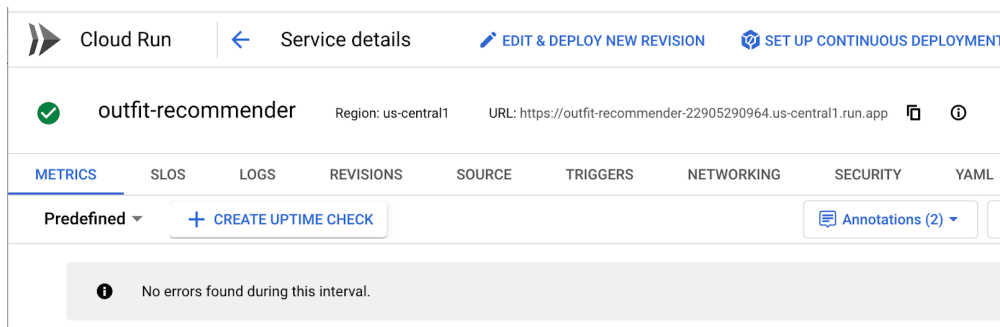
A service exposes a unique endpoint and automatically scales the underlying infrastructure to handle i
Deploy a container image, source code or a function to create a service.

Services

Filter Filter services

☐	☒	Name ↑	Deployment type	Req/sec ?	Region
☐	☒	outfit-recommender	(📁) Source	0	us-central1
☐	☒	retail-engine	(⋯) Function	0	us-central1

3. 按一下應用程式網址，開啟建議工具應用程式 UI。



Cloud Run Service details

EDIT & DEPLOY NEW REVISION SET UP CONTINUOUS DEPLOYMENT

✔ outfit-recommender Region: us-central1 URL: https://outfit-recommender-22905290964.us-central1.run.app

METRICS SLOS LOGS REVISIONS SOURCE TRIGGERS NETWORKING SECURITY YAML

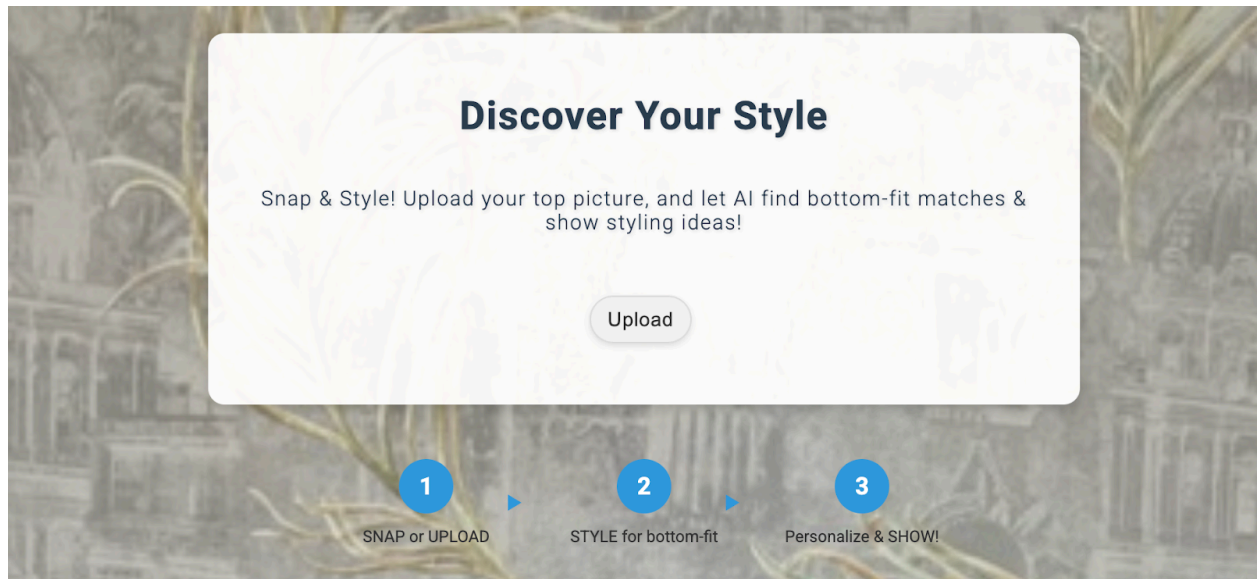
Predefined + CREATE UPTIME CHECK Annotations (2)

No errors found during this interval.

The following is a sample URL that you will use:

<https://outfit-recommender-22905290964.us-central1.run.app/style>

部署的應用程式如下所示：



使用應用程式

如要開始使用應用程式，請按照下列步驟操作：

1. 按一下「上傳」，然後上傳服飾圖片。
2. 上傳圖片後，按一下「樣式」。應用程式會將圖片做為提示，並根據零售引擎應用程式的提示生成底部選項，其中包含零售資料集的嵌入。

應用程式會根據圖片和風格建議，生成圖片建議和提示。例如：`A white semi-sheer button up blouse with pink floral patterns on it, with balloon sleeves.`

3. 您可以將其他提示傳遞至這項自動生成的樣式建議。例如：`STYLE RECOMMENDATION: Cute brown skirt on a curly updo. Make it photo realistic. Accessorize with cherry earrings and burgundy plastic case sling bag.`
4. 按一下「顯示」即可查看最終樣式。

A white semi-sheer button up blouse with pink floral patterns on it, with balloon sleeves. STYLE RECOMMENDATION: Cute brown skirt on a curly updo. Make it photo realistic. Accessorize with cherry earrings and burgundy plastic case sling bag.

****PRO TIP: Add personalization details for better styling visualization (on click of "show"). Example: A grey v-neck half sleeve t-shirt. STYLE RECOMMENDATION: A pair of cropped white trousers. Also add hoops for earrings and a stylish blunt bob cut for a hairstyle. Style for a wheatish brown complexioned, happy face woman.*



15. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-

步驟 16 · 約 0 分鐘

16. 恭喜

恭喜！您已成功使用 AlloyDB、pgvector 和 Vector Search 執行相似度搜尋，並搭配強大的 Imagen 模型使用搜尋結果，產生樣式建議。
